

勾股数元组

题目描述

如果3个正整数(a,b,c)满足 $a^2 + b^2 = c^2$ 的关系，则称(a,b,c)为勾股数（著名的勾三股四弦五），

为了探索勾股数的规律，我们定义如果勾股数(a,b,c)之间两两互质（即a与b，a与c，b与c之间均互质，没有公约数），则其为勾股数元组（例如(3,4,5)是勾股数元组，(6,8,10)则不是勾股数元组）。

请求出给定范围[N,M]内，所有的勾股数元组。

输入描述

起始范围N, $1 \leq N \leq 10000$

结束范围M, $N < M \leq 10000$

输出描述

- 1. a,b,c请保证 $a < b < c$,输出格式：a b c;
- 2. 多组勾股数元组请按照a升序，b升序，最后c升序的方式排序输出；
- 3. 给定范围中如果找不到勾股数元组时，输出”NA“。

用例

输入	1 20
输出	3 4 5 5 12 13

	8 15 17
说明	[1,20]范围内勾股数有：(3 4 5), (5 12 13), (6 8 10), (8 15 17), (9 12 15), (12 16 20); 其中，满足(a,b,c)之间两两互质的勾股数元组有：(3 4 5), (5 12 13), (8 15 17); 按输出描述中顺序要求输出结果。

输入	5 10
输出	NA
说明	[5,10]范围内勾股数有：(6 8 10); 其中，没有满足(a,b,c)之间两两互质的勾股数元组； 给定范围中找不到勾股数元组，输出"NA"

暴力枚举解法

本题首先需要找出给定区间内的所有勾股数，当找出勾股数后，继续判断勾股数两两之间是否互质，若否，则丢弃，若是，则保留。

最终保留的就是勾股数元组。

因此本题难点有二：1、如何找出所有勾股数； 2、如何判断两个数互质

关于1，我们可以先求出区间[n,m]的所有数的平方，缓存到一个数组arr中，然后对该数组进行双重for遍历，外层遍历所有元素arr[i]，内层遍历i之后的每一个元素arr[j]，我们求arr[i]+arr[j]的和sum，看arr中是否包含sum元素，若是，则就得到一组勾股数sqrt(arr[i])、sqrt(arr[j])、sqrt(sum)。

按照上面逻辑求得所有勾股数。

之后，我们可以根据 **辗转相除法** 判断两个数是否互质，比如求9和12是否互质，以及求47和18是否互质。

我们只需要用

$a \% b$ 得到一个 mod

然后将

$a \leq b$

$b \leq \text{mod}$

如果进行到 $b===0$ 时，则看此时a的值，若 $a===1$ ，则说明初始时的a,b互质，否则就有 **最大公约数** 结束时的a。

如下图所示：

	被除数	除数	余数			被除数	除数	余数		
	12	9	3			47	18	11		
	9	3	0			18	11	7		
	3	0				11	7	4		
						7	4	3		
						4	3	1		
						3	1	0		
						1	0			

CSDN @伏城之外

JS算法源码

```
1 | const rl = require("readline").createInterface({ input: process.stdin });
2 | var iter = rl[Symbol.asyncIterator]();
```

运行

```
3 | const readline = async () => (await iter.next()).value; 4 |
5 | void (async function () {
6 |     const n = parseInt(await readline());
7 |     const m = parseInt(await readline());
8 |
9 |     const arr = [];
10 |    for (let i = n; i <= m; i++) {
11 |        arr.push(i * i);
12 |    }
13 |
14 |    const set = new Set(arr);
15 |
16 |    const res = [];
17 |    for (let i = 0; i < arr.length; i++) {
18 |        for (let j = i + 1; j < arr.length; j++) {
19 |            const sum = arr[i] + arr[j];
20 |
21 |            if (set.has(sum)) {
22 |                const a = Math.sqrt(arr[i]);
23 |                const b = Math.sqrt(arr[j]);
24 |                const c = Math.sqrt(sum);
25 |
26 |                if (
27 |                    isRelativePrime(a, b) &&
28 |                    isRelativePrime(a, c) &&
29 |                    isRelativePrime(b, c)
30 |                ) {
31 |                    res.push([a, b, c]);
32 |                }
33 |            }
34 |        }
35 |    }
36 |
37 |    if (res.length > 0) {
38 |        res.forEach((group) => console.log(group.join(" ")));
39 |    } else {
40 |        console.log("NA");
    }
```

```

41 |     }
    |     42 | })();
43 |
44 | function isRelativePrime(x, y) {
45 |     while (y > 0) {
46 |         const mod = x % y;
47 |         x = y;
48 |         y = mod;
49 |     }
50 |
51 |     return x == 1;
52 | }

```

Java算法源码

```

1 | import java.util.*;
2 |
3 | public class Main {
4 |     public static void main(String[] args) {
5 |         Scanner sc = new Scanner(System.in);
6 |         int n = sc.nextInt();
7 |         int m = sc.nextInt();
8 |
9 |         Integer[] arr = new Integer[m - n + 1];
10 |         for (int i = n; i <= m; i++) {
11 |             arr[i - n] = i * i;
12 |         }
13 |
14 |         HashSet<Integer> set = new HashSet<>();
15 |         Collections.addAll(set, arr);
16 |
17 |         ArrayList<String> res = new ArrayList<>();
18 |         for (int i = 0; i < arr.length; i++) {
19 |             for (int j = i + 1; j < arr.length; j++) {

```

```

20         int sum = arr[i] + arr[j];
21     }
22     if (set.contains(sum)) {
23         int a = (int) Math.sqrt(arr[i]);
24         int b = (int) Math.sqrt(arr[j]);
25         int c = (int) Math.sqrt(sum);
26
27         if (isRelativePrime(a, b) && isRelativePrime(a, c) && isRelativePrime(b, c)) {
28             res.add(a + " " + b + " " + c);
29         }
30     }
31 }
32 }
33
34 if (res.isEmpty()) {
35     System.out.println("NA");
36 } else {
37     res.forEach(System.out::println);
38 }
39 }
40
41 // 判断两个数是否互质，辗转相除
42 public static boolean isRelativePrime(int x, int y) {
43     while (y > 0) {
44         int mod = x % y;
45         x = y;
46         y = mod;
47     }
48
49     return x == 1;
50 }
51 }

```

Python算法源码

```
1 import math 2 |
3 # 输入获取
4 n = int(input())
5 m = int(input())
6
7
8 # 判断两个数是否互质，辗转相除
9 def isRelativePrime(x, y):
10     while y > 0:
11         mod = x % y
12         x = y
13         y = mod
14
15     return x == 1
16
17
18 # 算法入口
19 def solution():
20     arr = []
21     for i in range(n, m + 1):
22         arr.append(i * i)
23
24     setArr = set(arr)
25
26     res = []
27     for i in range(len(arr)):
28         for j in range(i + 1, len(arr)):
29             # 判断勾股数  $a^2 + b^2 = c^2$ 
30             sumV = arr[i] + arr[j]
31             if sumV in setArr:
32                 a = int(math.sqrt(arr[i]))
33                 b = int(math.sqrt(arr[j]))
34                 c = int(math.sqrt(sumV))
35
36                 if isRelativePrime(a, b) and isRelativePrime(a, c) and isRelativePrime(b, c):
37                     res.append(f"{a} {b} {c}")
```

```
38 | 39 |     if len(res) == 0:
39 |     print("NA")
40 |
41 |     else:
42 |         for g in res:
43 |             print(g)
44 |
45 |
46 | # 算法调用
47 | solution()
```

C算法源码

```
1 | #include <stdio.h>
2 | #include <stdbool.h>
3 | #include <math.h>
4 |
5 | int binarySearch(const int arr[], int arr_size, int target) {
6 |     int low = 0;
7 |     int high = arr_size - 1;
8 |
9 |     while (low <= high) {
10 |         int mid = (low + high) >> 1;
11 |         int midVal = arr[mid];
12 |
13 |         if (midVal > target) {
14 |             high = mid - 1;
15 |         } else if (midVal < target) {
16 |             low = mid + 1;
17 |         } else {
18 |             return mid;
19 |         }
20 |     }
21 |
22 |     return -1;
23 | }
```



```

24 |
25 | bool isRelativePrime(int x, int y) {
26 |     while (y > 0) {
27 |         int mod = x % y;
28 |         x = y;
29 |         y = mod;
30 |     }
31 |
32 |     return x == 1;
33 | }
34 |
35 | int main() {
36 |     int n, m;
37 |     scanf("%d %d", &n, &m);
38 |
39 |     int arr_size = m - n + 1;
40 |
41 |     int arr[arr_size];
42 |     for (int i = n; i <= m; i++) {
43 |         arr[i - n] = i * i;
44 |     }
45 |
46 |     bool isFind = false;
47 |
48 |     for (int i = 0; i < arr_size; i++) {
49 |         for (int j = i + 1; j < arr_size; j++) {
50 |             if (binarySearch(arr, arr_size, arr[i] + arr[j]) != -1) {
51 |                 int a = (int) sqrt(arr[i]);
52 |                 int b = (int) sqrt(arr[j]);
53 |                 int c = (int) sqrt(arr[i] + arr[j]);
54 |
55 |                 if (isRelativePrime(a, b) && isRelativePrime(a, c) && isRelativePrime(b, c)) {
56 |                     isFind = true;
57 |                     printf("%d %d %d\n", a, b, c);
58 |                 }
59 |             }
60 |         }

```

```

61     }
62
63     if (!isFind) {
64         puts("NA");
65     }
66
67     return 0;
68 }

```

C++ 算法源码

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  bool isRelative(int x, int y) {
6      while (y > 0) {
7          int mod = x % y;
8          x = y;
9          y = mod;
10     }
11
12     return x == 1;
13 }
14
15 int main() {
16     int n, m;
17     cin >> n >> m;
18
19     vector<int> arr;
20     set<int> set;
21
22     for (int i = n; i <= m; i++) {
23         int val = i * i;
24         arr.emplace_back(val);

```

```

25         set.insert(val);26     }
27
28     bool isFind = false;
29
30     for (int i = 0; i < arr.size(); i++) {
31         for (int j = i + 1; j < arr.size(); j++) {
32             int sum = arr[i] + arr[j];
33
34             if (set.find(sum) != set.end()) {
35                 int a = (int) sqrt(arr[i]);
36                 int b = (int) sqrt(arr[j]);
37                 int c = (int) sqrt(sum);
38
39                 if (isRelative(a, b) && isRelative(a, c) && isRelative(b, c)) {
40                     isFind = true;
41                     cout << a << " " << b << " " << c << endl;
42                 }
43             }
44         }
45     }
46
47     if (!isFind) {
48         cout << "NA" << endl;
49     }
50
51     return 0;
52 }

```

费马大定理解法

互质勾股数组存在如下定理：

闲谈费马大定理 - 知乎 (zhihu.com)

从勾股数谈起

所谓勾股数 (Pythagorean triple)，指的是符合勾股定理 $x^2 + y^2 = z^2$ 的正整数数组 (x, y, z) 。因为任何一组勾股数的整数倍显然也是勾股数，所以一般而言，我们更加关注三者互质的数组 (x, y, z) 。这里我们将介绍一种产生勾股数的算法，并将看到互质的勾股数有无穷多组。

将勾股定理稍作改写，可以有

$$\begin{aligned}x^2 &= z^2 - y^2 = (z + y)(z - y) \\ \frac{z + y}{x} &= \frac{z - y}{x}\end{aligned}$$

令 $\frac{z+y}{x} = \frac{m}{n}$ ，其中 m, n 为满足 $m > n$ 的正整数。根据改写后的勾股关系，立刻有 $\frac{z-y}{x} = \frac{n}{m}$ 。我们进而可以写出如下关系：

$$\begin{cases} \frac{z}{x} + \frac{y}{x} = \frac{m}{n} \\ \frac{z}{x} - \frac{y}{x} = \frac{n}{m} \end{cases} \Rightarrow \begin{cases} \frac{z}{x} = \frac{1}{2} \left(\frac{m}{n} + \frac{n}{m} \right) = \frac{m^2 + n^2}{2mn} \\ \frac{y}{x} = \frac{1}{2} \left(\frac{m}{n} - \frac{n}{m} \right) = \frac{m^2 - n^2}{2mn} \end{cases}$$

很自然地，选取

$$x = 2mn, y = m^2 - n^2, z = m^2 + n^2 \quad (\#)$$

就会自动满足 $x^2 + y^2 = z^2$ 。

注意到，如此构造出来的 x 必为偶数，如果 m 和 n 同为奇数或者同为偶数，则 y 和 z 也将是偶数，这样 (x, y, z) 将含有公因子 2，不符合互质的要求。另一方面，如果 m 和 n 含有公因子 $p > 1$ ，那么 (x, y, z) 也将含有公因子 p ，也不符合互质的要求。

因此我们可以下结论：要通过 (#) 式生成互质的勾股数 (x, y, z) ， m 和 n 必须是一个奇偶数对，并且除了 1 之外它们没有其他公因子。但显然，这样的 (m, n) 有无穷多组，因此互质的勾股数也有无穷多组。

举点栗子，一些勾股数的构造如下：

$$\begin{aligned}m = 2, n = 1 &\Rightarrow (x, y, z) = (4, 3, 5) \\ m = 3, n = 2 &\Rightarrow (x, y, z) = (12, 5, 13) \\ m = 4, n = 1 &\Rightarrow (x, y, z) = (8, 15, 17) \\ \vdots &\end{aligned}$$

$$m = 4, n = 3 \Rightarrow (x, y, z) = (24, 7, 25)$$

而本题要在范围[n, m]中选取所有互质的勾股数元组，我们假设存在x, y, 且 (x > y) 可以推导出互质的勾股数三元组(a, b, c)

其中：

- $a = x^2 - y^2$
- $b = 2 * x * y$
- $c = x^2 + y^2$

而 $n \leq a, b < c \leq m$

因此，可以继续推导出x,y的取值范围：

$$a + c = 2 * x^2 \leq 2 * m$$

$$\text{即 } x \leq \sqrt{m}$$

$$c - a = 2 * y^2, \text{ 其中 } c > a, \text{ 因此 } c - a > 0$$

$$\text{即 } y > 0$$

而 $x > y$, 因此x,y取值范围是

$$0 < y < x \leq \sqrt{m}$$

另外，想要通过x,y推导出的勾股数三元组a,b,c互质，则x,y必然互质，且x,y必然一个是奇数，一个是偶数。

JS算法源码

```
1 | const rl = require("readline").createInterface({ input: process.stdin });
```

```

2 | var iter = rl[Symbol.asyncIterator]();
    | 3 | const readline = async () => (await iter.next()).value;
4 |
5 | void (async function () {
6 |     const n = parseInt(await readline());
7 |     const m = parseInt(await readline());
8 |
9 |     const ans = [];
10 |
11 |     const k = Math.ceil(Math.sqrt(m));
12 |
13 |     for (let y = 1; y < k; y++) {
14 |         for (let x = y + 1; x < k; x++) {
15 |             if (isRelativePrime(x, y) && (x + y) % 2 == 1) {
16 |                 const a = x * x - y * y;
17 |                 const b = 2 * x * y;
18 |                 const c = x * x + y * y;
19 |
20 |                 if (a >= n && b >= n && c <= m) {
21 |                     ans.push(a < b ? [a, b, c] : [b, a, c]);
22 |                 }
23 |             }
24 |         }
25 |     }
26 |
27 |     if (ans.length > 0) {
28 |         ans.sort((a, b) =>
29 |             a[0] != b[0] ? a[0] - b[0] : a[1] != b[1] ? a[1] - b[1] : a[2] - b[2]
30 |         );
31 |
32 |         for (let arr of ans) {
33 |             console.log(arr.join(" "));
34 |         }
35 |     } else {
36 |         console.log("NA");
37 |     }
38 | })();

```

```

39 |
40 | function isRelativePrime(x, y) {
41 |     while (y > 0) {
42 |         const mod = x % y;
43 |         x = y;
44 |         y = mod;
45 |     }
46 |
47 |     return x == 1;
48 | }

```

Java算法源码

```

1 | import java.util.ArrayList;
2 | import java.util.Scanner;
3 |
4 | public class Main {
5 |     public static void main(String[] args) {
6 |         Scanner sc = new Scanner(System.in);
7 |         int n = sc.nextInt();
8 |         int m = sc.nextInt();
9 |
10 |         ArrayList<int[]> ans = new ArrayList<>();
11 |
12 |         int k = (int) Math.ceil(Math.sqrt(m));
13 |
14 |         for (int y = 1; y < k; y++) {
15 |             for (int x = y + 1; x < k; x++) {
16 |                 if (isRelativePrime(x, y) && (x + y) % 2 == 1) {
17 |                     int a = x * x - y * y;
18 |                     int b = 2 * x * y;
19 |                     int c = x * x + y * y;
20 |
21 |                     if (a >= n && b >= n && c <= m) {

```

```

22         ans.add(a < b ? new int[]{a, b, c} : new int[]{b, a, c});
23     }
24     }
25 }
26 }
27
28 if (!ans.isEmpty()) {
29     ans.sort((a, b) -> a[0] != b[0] ? a[0] - b[0] : a[1] != b[1] ? a[1] - b[1] : a[2] - b[2]);
30     for (int[] group : ans) {
31         System.out.println(group[0] + " " + group[1] + " " + group[2]);
32     }
33 } else {
34     System.out.println("NA");
35 }
36 }
37
38 // 判断两个数是否互质，辗转相除
39 public static boolean isRelativePrime(int x, int y) {
40     while (y > 0) {
41         int mod = x % y;
42         x = y;
43         y = mod;
44     }
45
46     return x == 1;
47 }
48 }

```

Python算法源码

```

1 import math
2
3 # 输入获取
4 n = int(input())

```



```
5 | m = int(input()) 6 |
7 |
8 | # 判断两个数是否互质，辗转相除
9 | def isRelativePrime(x, y):
10 |     while y > 0:
11 |         mod = x % y
12 |         x = y
13 |         y = mod
14 |
15 |     return x == 1
16 |
17 |
18 | # 算法入口
19 | def getResult():
20 |     ans = []
21 |
22 |     k = math.ceil(math.sqrt(m))
23 |
24 |     for y in range(1, k):
25 |         for x in range(y + 1, k):
26 |             # 互质勾股数
27 |             if isRelativePrime(x, y) and (x + y) % 2 == 1:
28 |                 a = x * x - y * y
29 |                 b = 2 * x * y
30 |                 c = x * x + y * y
31 |
32 |                 if a >= n and b >= n and c <= m:
33 |                     ans.append([a, b, c] if a < b else [b, a, c])
34 |
35 |     if len(ans) > 0:
36 |         ans.sort()
37 |         for lst in ans:
38 |             print(" ".join(map(str, lst)))
39 |     else:
40 |         print("NA")
41 |
```

```
42 | 43 | # 算法调用
44 | getResult()
```

C算法源码

```
1  #include <stdio.h>
2  #include <stdbool.h>
3  #include <math.h>
4  #include <stdlib.h>
5
6  bool isRelativePrime(int x, int y) {
7      while (y > 0) {
8          int mod = x % y;
9          x = y;
10         y = mod;
11     }
12
13     return x == 1;
14 }
15
16 int cmp(const void *a, const void *b) {
17     int *A = (int *) a;
18     int *B = (int *) b;
19     return A[0] != B[0] ? A[0] - B[0] : A[1] != B[1] ? A[1] - B[1] : A[2] - B[2];
20 }
21
22 int main() {
23     int n, m;
24     scanf("%d %d", &n, &m);
25
26     int res[2000][3];
27     int res_size = 0;
28
29     int k = (int) ceil(sqrt(m));
30 }
```

```

31 |     for (int y = 1; y < k; y++) {
32 |         for (int x = y + 1; x < k; x++) {
33 |             if (isRelativePrime(x, y) && (x + y) % 2 == 1) {
34 |                 int a = x * x - y * y;
35 |                 int b = 2 * x * y;
36 |                 int c = x * x + y * y;
37 |
38 |                 if (a >= n && b >= n && c <= m) {
39 |                     res[res_size][0] = (int) fmin(a, b);
40 |                     res[res_size][1] = (int) fmax(a, b);
41 |                     res[res_size][2] = c;
42 |                     res_size++;
43 |                 }
44 |             }
45 |         }
46 |     }
47 |
48 |     if (res_size > 0) {
49 |         qsort(res, res_size, sizeof(res[0]), cmp);
50 |
51 |         for (int i = 0; i < res_size; i++) {
52 |             printf("%d %d %d\n", res[i][0], res[i][1], res[i][2]);
53 |         }
54 |     } else {
55 |         puts("NA");
56 |     }
57 |
58 |     return 0;
59 | }

```

C++ 算法源码

```

1 | #include <bits/stdc++.h>
2 |

```

```

3 using namespace std;
4
5 bool isRelative(int x, int y) {
6     while (y > 0) {
7         int mod = x % y;
8         x = y;
9         y = mod;
10    }
11
12    return x == 1;
13 }
14
15 int main() {
16     int n, m;
17     cin >> n >> m;
18
19     vector<vector<int>> res;
20
21     int k = (int) ceil(sqrt(m));
22
23     for (int y = 1; y < k; y++) {
24         for (int x = y + 1; x < k; x++) {
25             if (isRelative(x, y) && (x + y) % 2 == 1) {
26                 int a = x * x - y * y;
27                 int b = 2 * x * y;
28                 int c = x * x + y * y;
29
30                 if (a >= n && b >= n && c <= m) {
31                     res.emplace_back(vector<int>{min(a, b), max(a, b), c});
32                 }
33             }
34         }
35     }
36
37     if (!res.empty()) {
38         sort(res.begin(), res.end(), [](vector<int> &a, vector<int> &b) {
39             return a[0] != b[0] ? a[0] < b[0] : a[1] != b[1] ? a[1] < b[1] : a[2] < b[2];

```

```
40     });41
42     for (vector<int> &group: res) {
43         cout << group[0] << " " << group[1] << " " << group[2] << endl;
44     }
45 } else {
46     cout << "NA" << endl;
47 }
48
49 return 0;
50 }
```